

Raspberry Pi TeXNote P1 認証サーバー構築手順

ユーザー登録から JWT 認証、P2 中継、利用制限まで

2026 年 8 月 23 日

概要

本書は、Raspberry Pi 上で実際に行った P1 認証サーバーの構築・動作確認記録と、`Server/P1/progs` に保存された現行ソースを一つにまとめた再構築手順書である。SQLite と Argon2 によるユーザー認証、HS256 JWT、保護 API、P2 TeX サーバーへの中継、利用履歴、プラン別月間回数制限を扱う。掲載する現行ソースは文書コンパイル時に実ファイルから直接取り込むため、説明とソースの食い違いを確認しやすい。

目次

1	構成と設計方針	2
1.1	2 台の Raspberry Pi の役割	2
1.2	2 種類の秘密値	2
2	Raspberry Pi の準備	2
2.1	P1 への接続と状態確認	2
2.2	Python 環境	3
3	SQLite データベースの作成	3
3.1	現行ファイル構成	3
3.2	テーブル設計	4
3.3	<code>init.db.py</code>	4
3.4	<code>database.py</code>	5
3.5	<code>plans.py</code>	6
4	ユーザー登録とパスワード認証	6
4.1	<code>auth.py</code>	6
4.2	<code>useradd.py</code>	7
4.3	<code>change_plan.py</code>	8
4.4	<code>check01.py</code> と <code>check02.py</code>	9
5	JWT と FastAPI 公開 API	10
5.1	環境変数	10
5.2	<code>main.py</code> の構成	10
5.3	起動	15
6	ログインと保護 API の試験	15
6.1	POST <code>/login</code>	15
6.2	GET <code>/me</code>	16

7	P2 との疎通と内部認証	16
7.1	ネットワークとヘルスチェック	16
7.2	P1-P2 固定トークン	16
8	コンパイル中継の段階的試験	16
8.1	認証だけの compile-test	16
8.2	現行 compile-test	17
8.3	POST /compile	17
8.4	P2 エラー詳細の中継	17
9	利用履歴と月間回数制限	17
10	現行ソースを配置する方法	19
11	ソース整理で改善した点と残る課題	19
11.1	main.py の整理と修正	19
11.2	その他の Python ソースの整理	19
11.3	運転用と保守用の分離	20
11.4	公開前に追加する制限	20
12	systemd による常時起動	20
12.1	Python 環境の確認	20
12.2	手動起動の確認	20
12.3	配置と権限の確認	21
12.4	秘密値ファイルの作成	21
12.5	サービスファイルの作成	21
12.6	設定検査と自動起動の有効化	22
12.7	API と再起動後の確認	22
12.8	運用時の操作	22
13	Docker 版への移行	24
13.1	移行方針	24
13.2	SQLite 保存先を環境変数化する	24
13.3	Docker Engine と Compose の確認	24
13.4	Docker 用ディレクトリの準備	24
13.5	requirements.txt の作成	25
13.6	Dockerfile の作成	25
13.7	秘密情報をビルド対象から除外する	26
13.8	Docker 用環境ファイルの作成	26
13.9	Compose 設定の作成	26
13.10	イメージのビルドと静的検査	27
13.11	systemd 版 DB の移行	27
13.12	systemd 版から Docker 版への切替	28
13.13	認証と P2 中継の確認	28
13.14	再起動後の自動復旧試験	29
13.15	更新と日常運用	29
13.16	DB のバックアップ	29

13.17	問題発生時に systemd 版へ戻す	30
13.18	Docker 版完了チェックリスト	30
14	セキュリティと公開前チェック	30
15	再構築時の確認順序	30
16	まとめ	31

1 構成と設計方針

1.1 2 台の Raspberry Pi の役割

機器	記録上のアドレス	役割	主な API
P1	192.168.3.21	外部向け認証・API 中継	/login, /me, /compile
P2	192.168.3.4:8000	内部 TeX コンパイル	/health, /v1/auth-check, /v1/typeset

外部ユーザーは P1 だけへ接続し、P2 へ直接アクセスしない。P1 はユーザー JWT を検証し、プランと利用回数を確認した後、P1-P2 専用固定トークンを付けて P2 へ中継する。

```
User -- email/password --> P1 /login
User <-- USER_JWT ----- P1

User -- Bearer USER_JWT --> P1 /compile
                        | user/status/plan/usage を確認
                        | Bearer TEXNOTE_API_TOKEN
                        v
                        P2 /v1/typeset
                        |
                        v
User <-- PDF Base64 + log -- P1
```

1.2 2 種類の秘密値

環境変数	使用区間	用途
TEX_AUTH_SECRET_KEY	User-P1	JWT の署名・検証
TEXNOTE_API_TOKEN	P1-P2	P2 内部 API の Bearer 認証

両者を同じ値にしない。秘密値の実値、ユーザーパスワード、発行済み JWT、DB 本体を Git へ保存しない。

2 Raspberry Pi の準備

2.1 P1 への接続と状態確認

Mac から、実際の操作記録で使った P1 へ SSH 接続する。

```
ssh mat@192.168.3.21
```

P1 上で OS、CPU、ストレージ、時刻、更新状態を確認する。

```
cat /etc/os-release
uname -m
df -h
hostname -I
tr -d '\0' </proc/device-tree/model; echo
```

```
free -h
hostnamectl
timedatectl
sudo apt update
apt list --upgradable
test -f /var/run/reboot-required \
    && cat /var/run/reboot-required \
    || echo "再起動は要求されていません"
```

必要な場合だけ更新して再起動する。

```
sudo apt full-upgrade
sudo reboot
```

P1 と P2 の IP が変わると接続できなくなるため、ルーターの DHCP 予約を設定する。

2.2 Python 環境

実際の作業ディレクトリ~/tex-auth と仮想環境.auth-env を作る。

```
sudo apt install python3 python3-venv sqlite3
mkdir -p ~/tex-auth
cd ~/tex-auth
python3 -m venv .auth-env
source .auth-env/bin/activate
python -m pip install --upgrade pip
python -m pip install \
    fastapi \
    'uvicorn[standard]' \
    pyjwt \
    argon2-cffi \
    requests
```

動作確認した依存バージョンを保存する。

```
python -m pip freeze > requirements.txt
```

3 SQLite データベースの作成

3.1 現行ファイル構成

```
tex-auth/
|-- database.py
|-- auth.py
|-- plans.py
|-- main.py
|-- tools/
|   |-- __init__.py
|   |-- init_db.py
|   |-- useradd.py
|   |-- change_plan.py
|   |-- check01.py
|   |-- check02.py
```

```
|  `-- show_usage.py
`-- tex_service.db      # init_db.py 実行後に生成
```

tex_service.db は認証情報を含む実データであり、Git や一般公開する文書へ含めない。ルート直下の 4 つの Python ファイルがサーバー運転用であり、tools は初期構築・管理・確認時だけ使用する。

3.2 テーブル設計

表	主な列	目的
users	id, email, password_hash, status, plan, created_at	ユーザー認証とプラン
api_keys	id, user_id, key_hash, created_at, revoked_at	将来の利用者別 API キー
usage	id, user_id, created_at, com- pile_seconds, input_bytes, success	利用履歴と月間集計

3.3 init_db.py

tools/init_db.py の現行ソース全文をリスト 1 に示す。

ソースコード 1 tools/init_db.py：データベース初期化

```

1  """P1が使用するSQLiteデータベースとテーブルを初期化する。"""
2
3  import sqlite3
4
5  from database import DB_PATH
6
7
8  def create_database() -> None:
9      """必要な3テーブルを、存在しない場合だけ作成する。"""
10     connection = sqlite3.connect(DB_PATH)
11
12     try:
13         cursor = connection.cursor()
14         cursor.execute("PRAGMA foreign_keys = ON")
15
16         # ユーザー認証、状態、契約プランを保持する。
17         cursor.execute(
18             """
19             CREATE TABLE IF NOT EXISTS users (
20                 id INTEGER PRIMARY KEY AUTOINCREMENT,
21                 email TEXT NOT NULL UNIQUE,
22                 password_hash TEXT NOT NULL,
23                 status TEXT NOT NULL DEFAULT 'active',
24                 plan TEXT NOT NULL DEFAULT 'free',
25                 created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
26             )
27             """
28         )
29
30         # 将来の利用者別APIキー発行・失効管理に使用する。
31         cursor.execute(
32             """
33             CREATE TABLE IF NOT EXISTS api_keys (
34                 id INTEGER PRIMARY KEY AUTOINCREMENT,
35                 user_id INTEGER NOT NULL,
36                 key_hash TEXT NOT NULL UNIQUE,
37                 created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
38                 revoked_at TEXT,
39                 FOREIGN KEY (user_id) REFERENCES users(id)
40                 ON DELETE CASCADE
41             )
42             """
43         )
44
45         # コンパイル要求の所要時間、入力サイズ、成否を記録する。
46         cursor.execute(

```

```

47         """
48         CREATE TABLE IF NOT EXISTS usage (
49             id INTEGER PRIMARY KEY AUTOINCREMENT,
50             user_id INTEGER NOT NULL,
51             created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
52             compile_seconds REAL NOT NULL DEFAULT 0,
53             input_bytes INTEGER NOT NULL DEFAULT 0,
54             success INTEGER NOT NULL DEFAULT 0,
55             FOREIGN KEY (user_id) REFERENCES users(id)
56             ON DELETE CASCADE
57         )
58         """
59     )
60
61     connection.commit()
62 finally:
63     connection.close()
64
65
66 def main() -> None:
67     """コマンドとしてDBを初期化し、作成先を表示する。"""
68     create_database()
69     print(f"Database initialized: {DB_PATH}")
70
71
72 if __name__ == "__main__":
73     main()

```

database.py の DB_PATH を共用するため、初期化と運転時で同じ DB を確実に参照する。保守ツールは~/tex-auth からモジュールとして実行する。

初期化し、表を確認する。

```

cd ~/tex-auth
python -m tools.init_db
sqlite3 tex_service.db '.tables'
sqlite3 tex_service.db '.schema'
chmod 600 tex_service.db

```

3.4 database.py

現行の database.py 全文をリスト 2 に示す。

ソースコード 2 database.py : SQLite 接続

```

1  """TeXNote P1で使用するSQLite接続を提供する。
2
3  DBファイルの位置と接続時の共通設定をこのモジュールへ集約し、
4  運転用コードと保守ツールが必ず同じデータベースを参照するようにする。
5  """
6
7  from collections.abc import Iterator
8  from contextlib import contextmanager
9  from pathlib import Path
10 import sqlite3
11
12
13 # このファイルと同じディレクトリにP1のDBを配置する。
14 DB_PATH = Path(__file__).resolve().parent / "tex_service.db"
15
16
17 @contextmanager
18 def get_connection() -> Iterator[sqlite3.Connection]:
19     """共通設定済みのDB接続を開き、処理後に必ず閉じる。"""
20     connection = sqlite3.connect(DB_PATH)
21
22     # SELECT結果を row["email"] のように列名で参照できるようにする。
23     connection.row_factory = sqlite3.Row
24
25     # SQLiteの外部キー制約は接続ごとに有効化が必要がある。
26     connection.execute("PRAGMA foreign_keys = ON")
27
28     try:
29         # withブロックが正常終了した場合はコミットし、例外時はロールバックする。
30         with connection:
31             yield connection
32     finally:
33         connection.close()

```

`Path(__file__).parent` を基準にするため、起動ディレクトリに依存せず同じ DB を開ける。`sqlite3.Row` で列名アクセスを可能にし、接続ごとに外部キー制約を有効にする。`get_connection()` はコンテキストマネージャーであり、正常時のコミット、例外時のロールバック、最後の接続クローズを一箇所で保証する。

3.5 plans.py

利用可能な plan と月間上限をリスト 3 の一箇所で定義する。

ソースコード 3 plans.py：利用プランと月間上限

```
1  """ P1 が受け付ける利用プランと月間コンパイル上限を定義する。 """
2
3
4  PLAN_MONTHLY_LIMITS = {
5      "free": 50,
6      "standard": 1_000,
7      "pro": 5_000,
8  }
9
10
11 def validated_plan(value: str) -> str:
12     """正規化したプラン名を返し、未定義値は拒否する。"""
13     plan = value.strip().lower()
14     if plan not in PLAN_MONTHLY_LIMITS:
15         allowed = ", ".join(PLAN_MONTHLY_LIMITS)
16         raise ValueError(f"Plan must be one of: {allowed}")
17     return plan
```

`main.py` の上限判定、ユーザー登録、プラン変更は同じ定義を参照する。入力値は前後の空白を除去して小文字化し、`free`、`standard`、`pro` 以外を DB へ保存しない。

4 ユーザー登録とパスワード認証

4.1 auth.py

現行の `auth.py` 全文をリスト 4 に示す。

ソースコード 4 auth.py：Argon2 ユーザー認証

```
1  """ TeXNote P1 のユーザー登録とパスワード認証を提供する。 """
2
3  import sqlite3
4
5  from argon2 import PasswordHasher
6  from argon2.exceptions import VerifyMismatchError
7
8  from database import get_connection
9  from plans import validated_plan
10
11
12 # Argon2 の安全な既定パラメーターでハッシュ生成・照合を行う。
13 password_hasher = PasswordHasher()
14
15
16 def create_user(email: str, password: str, plan: str = "free") -> int:
17     """パスワードを Argon2 でハッシュ化し、新しいユーザーを登録する。"""
18     plan = validated_plan(plan)
19     password_hash = password_hasher.hash(password)
20
21     with get_connection() as connection:
22         cursor = connection.execute(
23             """
24             INSERT INTO users (
25                 email,
26                 password_hash,
27                 plan
28             )
29             VALUES (?, ?, ?)
30             """
31             (email, password_hash, plan),
32         )
33         user_id = cursor.lastrowid
34
35 # AUTOINCREMENT で発行された ID は通常 int だが、型上は None の可能性がある。
36 if user_id is None:
37     raise sqlite3.DatabaseError("Failed to obtain the new user ID")
```



```

38
39     return int(user_id)
40
41
42 def authenticate_user(email: str, password: str) -> sqlite3.Row | None:
43     """有効なユーザーのパスワードが一致すれば、そのDB行を返す。"""
44     with get_connection() as connection:
45         user = connection.execute(
46             """
47             SELECT *
48             FROM users
49             WHERE email = ?
50             """,
51             (email,),
52         ).fetchone()
53
54     # ユーザー不存在とパスワード不一致は呼び出し側で同じ認証失敗として扱う。
55     if user is None:
56         return None
57
58     # 発行済みJWTがあっても、停止ユーザーは新たにログインさせない。
59     if user["status"] != "active":
60         return None
61
62     try:
63         password_hasher.verify(user["password_hash"], password)
64     except VerifyMismatchError:
65         return None
66
67     return user

```

平文パスワードをDBへ保存せず、Argon2 ハッシュだけを保存する。認証時は email 検索、ユーザー存在、status == active、Argon2 照合を順に確認する。SQL には必ず?プレースホルダーを用い、入力値を文字列連結しない。

4.2 useradd.py

現行ソース全文を示す。

ソースコード 5 tools/useradd.py: 対話式ユーザー登録

```

1  """管理者が対話操作でPIユーザーを追加するための保守ツール。"""
2
3  from getpass import getpass
4  from sqlite3 import IntegrityError
5
6  from auth import create_user
7  from plans import validated_plan
8
9
10 DEFAULT_PLAN = "free"
11
12
13 def prompt_user_data() -> tuple[str, str, str]:
14     """メールアドレス、プラン、確認済みパスワードを対話入力する。"""
15     email = input("Email: ").strip()
16     plan = input(f"Plan [{DEFAULT_PLAN}]: ").strip() or DEFAULT_PLAN
17     password = getpass("Password: ")
18     password_again = getpass("Password (again): ")
19
20     if not email:
21         raise SystemExit("Email is required")
22     if not password:
23         raise SystemExit("Password is required")
24     if password != password_again:
25         raise SystemExit("Passwords do not match")
26
27     try:
28         plan = validated_plan(plan)
29     except ValueError as error:
30         raise SystemExit(str(error)) from error
31
32     return email, password, plan
33
34
35 def main() -> None:
36     """入力内容でユーザーを登録し、発行されたIDを表示する。"""
37     email, password, plan = prompt_user_data()
38
39     try:
40         user_id = create_user(email, password, plan)
41     except IntegrityError as error:

```

```

42     # users.emailのUNIQUE制約違反を管理者向けの表示へ変換する。
43     raise SystemExit("Email is already registered") from error
44
45     print(f"User created: id={user_id}, email={email}, plan={plan}")
46
47
48 if __name__ == "__main__":
49     main()

```

現行版はメールアドレスと plan を標準入力、パスワードを `getpass` で 2 回入力する。パスワードは画面、ソースコード、シェル履歴に残らない。重複メールや入力不一致も明示的なメッセージで終了する。

```

cd ~/tex-auth
source .auth-env/bin/activate
python -m tools.useradd

```

同じメールアドレスには UNIQUE 制約があるため、同じ DB で再実行すると登録エラーになる。未定義 plan を入力した場合も登録せず、許可された plan を表示して終了する。

4.3 change_plan.py

既存ユーザーの plan は管理者用コマンドで変更する。現行ソース全文を示す。

ソースコード 6 tools/change_plan.py: 既存ユーザーのプラン変更

```

1  """管理者が既存 P1 ユーザーの利用プランを変更するための保守ツール。"""
2
3  import argparse
4
5  from database import get_connection
6  from plans import PLAN_MONTHLY_LIMITS, validated_plan
7
8
9  def change_plan(email: str, requested_plan: str) -> tuple[str, str]:
10     """ユーザーのプランを変更し、変更前と変更後の値を返す。"""
11     normalized_email = email.strip()
12     if not normalized_email:
13         raise ValueError("Email is required")
14     plan = validated_plan(requested_plan)
15
16     with get_connection() as connection:
17         user = connection.execute(
18             "SELECT plan FROM users WHERE email = ?",
19             (normalized_email,),
20         ).fetchone()
21         if user is None:
22             raise LookupError("User not found")
23
24         previous_plan = str(user["plan"])
25         connection.execute(
26             "UPDATE users SET plan = ? WHERE email = ?",
27             (plan, normalized_email),
28         )
29
30     return previous_plan, plan
31
32
33 def parse_arguments() -> argparse.Namespace:
34     """メールアドレスと変更先プランを解析する。"""
35     parser = argparse.ArgumentParser(description=__doc__)
36     parser.add_argument("email", help="変更するユーザーのメールアドレス")
37     parser.add_argument(
38         "plan",
39         choices=tuple(PLAN_MONTHLY_LIMITS),
40         help="変更後の利用プラン",
41     )
42     return parser.parse_args()
43
44
45 def main() -> None:
46     """指定ユーザーのプランを変更し、結果を表示する。"""
47     arguments = parse_arguments()
48     try:
49         previous_plan, plan = change_plan(arguments.email, arguments.plan)
50     except (LookupError, ValueError) as error:
51         raise SystemExit(str(error)) from error
52
53     print(
54         "Plan changed: "

```

```

55         f"email={arguments.email.strip()}, "
56         f"{previous_plan} -> {plan}"
57     )
58
59
60 if __name__ == "__main__":
61     main()

```

P1 上の運転用ディレクトリで、メールアドレスと変更先 plan を指定する。

```

cd ~/tex-auth
source .auth-env/bin/activate
python -m tools.change_plan user@example.com standard
python -m tools.check02 user@example.com

```

変更先は free、standard、pro のいずれかに限定される。ユーザーが存在しない場合は更新せずエラー終了する。成功時は変更前と変更後の plan を表示し、続けて check02 で DB の結果を確認する。

4.4 check01.py と check02.py

check01.py は users 表の一覧、check02.py は指定したテストユーザーを列名アクセスで確認する補助スクリプトである。現行ソースを示す。

ソースコード 7 tools/check01.py：ユーザー一覧確認

```

1  """usersテーブルのユーザー一覧を表示する確認ツール。"""
2
3  import sqlite3
4
5  from database import get_connection
6
7
8  def fetch_users() -> list[sqlite3.Row]:
9      """登録順に、パスワードハッシュを除いたユーザー情報を取得する。"""
10     with get_connection() as connection:
11         return connection.execute(
12             """
13             SELECT id, email, status, plan, created_at
14             FROM users
15             ORDER BY id
16             """
17         ).fetchall()
18
19
20 def main() -> None:
21     """全ユーザーを1行ずつ辞書形式で表示する。"""
22     for user in fetch_users():
23         print(dict(user))
24
25
26 if __name__ == "__main__":
27     main()

```

ソースコード 8 tools/check02.py：指定ユーザー確認

```

1  """メールアドレスを指定して1ユーザーを確認するツール。"""
2
3  import argparse
4  import sqlite3
5
6  from database import get_connection
7
8
9  def find_user(email: str) -> sqlite3.Row | None:
10     """パスワードハッシュを除いたユーザー情報をメールアドレスで検索する。"""
11     with get_connection() as connection:
12         return connection.execute(
13             """
14             SELECT id, email, status, plan, created_at
15             FROM users
16             WHERE email = ?
17             """
18         ).fetchone(
19             (email,)
20         )
21

```

```

22 def parse_arguments() -> argparse.Namespace:
23     """コマンドライン引数を解析する。"""
24     parser = argparse.ArgumentParser(description=__doc__)
25     parser.add_argument("email", help="確認するユーザーのメールアドレス")
26     return parser.parse_args()
27
28
29 def main() -> None:
30     """指定ユーザーを表示し、存在しなければエラー終了する。"""
31     arguments = parse_arguments()
32     user = find_user(arguments.email)
33
34     if user is None:
35         raise SystemExit("User not found")
36
37     print(dict(user))
38
39
40 if __name__ == "__main__":
41     main()

```

check02.py は確認するメールアドレスをコマンド引数で受け取る。

```

cd ~/tex-auth
python -m tools.check01
python -m tools.check02 test@example.com

```

パスワード照合は後述の POST /login で、正しい値と誤った値の両方を試験する。

5 JWT と FastAPI 公開 API

5.1 環境変数

JWT 署名鍵を生成し、P2 と共有する内部トークンを P1 へ設定する。

```

export TEX_AUTH_SECRET_KEY="$(python3 -c \
    'import secrets; print(secrets.token_hex(32))')"
export TEXNOTE_API_TOKEN='<P2 に設定したものと同一内部トークン>'
# P2 のアドレスを変更する場合だけ設定する
# export TEXNOTE_P2_TYPESET_URL='http://192.168.3.4:8000/v1/typeset'

```

`$(...)` はコマンド置換であり、Python が出力したランダム文字列を環境変数へ代入する。確認時も秘密値そのものを画面共有やログに残さない。

5.2 main.py の構成

現行の main.py 全文をリスト 9 に示す。このファイルには JWT、/login、/me、/compile-test、/compile、usage 記録、月間制限が実装されている。

ソースコード 9 main.py : P1 FastAPI 現行実装

```

1  """TeXNote P1 認証・API中継サーバー。
2
3  外部ユーザーの認証と利用制限を担当し、認証済みのコンパイル要求だけを
4  内部ネットワーク上のP2 TeXサーバーへ転送する。
5  """
6
7  import os
8  import time
9  import uuid
10 from datetime import datetime, timedelta, timezone
11 from typing import Literal
12
13 import jwt
14 import requests
15 from fastapi import Depends, FastAPI, HTTPException, Request, status
16 from fastapi.responses import JSONResponse
17 from fastapi.security import OAuth2PasswordBearer
18 from jwt.exceptions import InvalidTokenError

```

```

19 from pydantic import BaseModel, Field
20
21 from auth import authenticate_user
22 from database import get_connection
23 from plans import PLAN_MONTHLY_LIMITS
24
25
26 # -----
27 # アプリケーション設定
28 # -----
29
30 app = FastAPI(
31     title="TeXNote Authentication API",
32     version="1.0.0",
33 )
34
35 # 秘密値はソースコードへ書かず、起動環境から受け取る。
36 SECRET_KEY = os.environ["TEX_AUTH_SECRET_KEY"]
37 P2_API_TOKEN = os.environ["TEXNOTE_API_TOKEN"]
38
39 JWT_ALGORITHM = "HS256"
40 ACCESS_TOKEN_EXPIRE_MINUTES = 30
41 MAX_REQUEST_BYTES = 32 * 1024 * 1024
42
43 # P2の移設に備えて環境変数で上書きできるようにし、現行値を既定値とする。
44 P2_TYPESET_URL = os.getenv(
45     "TEXNOTE_P2_TYPESET_URL",
46     "http://192.168.3.4:8000/v1/typeset",
47 )
48 P2_REQUEST_TIMEOUT_SECONDS = 30
49
50 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")
51 SERVICE_NAME = "texnote"
52
53
54 # -----
55 # HTTPリクエスト制限
56 # -----
57
58 @app.middleware("http")
59 async def limit_request_size(request: Request, call_next):
60     """Content-Lengthが32 MiBを超える要求を処理前に拒否する。"""
61     content_length = request.headers.get("content-length")
62     if content_length is not None:
63         try:
64             request_bytes = int(content_length)
65         except ValueError:
66             return JSONResponse(
67                 status_code=status.HTTP_400_BAD_REQUEST,
68                 content={"detail": "Content-Lengthが不正です。"},
69             )
70
71     if request_bytes > MAX_REQUEST_BYTES:
72         return JSONResponse(
73             status_code=status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
74             content={
75                 "detail": "リクエストの上限32 MiBを超えています。"
76             },
77         )
78
79     return await call_next(request)
80
81
82 # -----
83 # API入出力モデル
84 # -----
85
86 class LoginRequest(BaseModel):
87     email: str
88     password: str
89     service: Literal["texnote"]
90
91
92 class FileData(BaseModel):
93     relativePath: str
94     data: str
95
96
97 class CompileRequest(BaseModel):
98     engine: Literal[
99         "lualatex",
100         "xelatex",
101         "pdflatex",
102         "uplatex",
103         "platex",
104     ] = "lualatex"
105     source: str
106     pictures: list[FileData] = Field(default_factory=list)
107     files: list[FileData] = Field(default_factory=list)

```

```

108
109
110 # -----
111 # JWTとユーザー認証
112 # -----
113
114 def create_access_token(user_id: int, email: str) -> str:
115     """ユーザー IDとメールアドレスを含む短寿命JWTを発行する。"""
116     now = datetime.now(timezone.utc)
117     payload = {
118         "sub": str(user_id),
119         "email": email,
120         "aud": SERVICE_NAME,
121         "iat": now,
122         "exp": now + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),
123     }
124     return jwt.encode(payload, SECRET_KEY, algorithm=JWT_ALGORITHM)
125
126
127 def unauthorized(detail: str) -> HTTPException:
128     """Bearer認証用ヘッダーを含む401例外を生成する。"""
129     return HTTPException(
130         status_code=status.HTTP_401_UNAUTHORIZED,
131         detail=detail,
132         headers={"WWW-Authenticate": "Bearer"},
133     )
134
135
136 def get_current_user(token: str = Depends(oauth2_scheme)):
137     """JWTを検証し、現在も有効なユーザーをDBから取得する。"""
138     try:
139         payload = jwt.decode(
140             token,
141             SECRET_KEY,
142             algorithms=[JWT_ALGORITHM],
143             audience=SERVICE_NAME,
144         )
145         user_id = int(payload["sub"])
146     except (InvalidTokenError, KeyError, TypeError, ValueError):
147         raise unauthorized("Invalid or expired token")
148
149     with get_connection() as con:
150         user = con.execute(
151             "SELECT * FROM users WHERE id = ?",
152             (user_id,),
153         ).fetchone()
154
155     if user is None:
156         raise unauthorized("User not found")
157
158     if user["status"] != "active":
159         raise HTTPException(
160             status_code=status.HTTP_403_FORBIDDEN,
161             detail="User is inactive",
162         )
163
164     return user
165
166
167 # -----
168 # 利用履歴と月間制限
169 # -----
170
171 def get_monthly_usage_count(user_id: int) -> int:
172     """当月に受け付けたコンパイル要求数（成功・失敗の両方）を返す。"""
173     with get_connection() as con:
174         row = con.execute(
175             """
176             SELECT COUNT(*) AS count
177             FROM usage
178             WHERE user_id = ?
179                 AND strftime('%Y-%m', created_at) =
180                     strftime('%Y-%m', 'now')
181             """,
182             (user_id,),
183         ).fetchone()
184
185     return int(row["count"])
186
187
188 def record_usage(
189     user_id: int,
190     compile_seconds: float,
191     input_bytes: int,
192     success: bool,
193 ) -> None:
194     """コンパイル要求の所要時間、入力サイズ、成否を記録する。"""
195     with get_connection() as con:
196         con.execute(

```

```

197         """
198         INSERT INTO usage (
199             user_id,
200             compile_seconds,
201             input_bytes,
202             success
203         )
204         VALUES (?, ?, ?, ?)
205         """
206         (
207             user_id,
208             compile_seconds,
209             input_bytes,
210             1 if success else 0,
211         ),
212     )
213
214
215 def ensure_monthly_limit(user) -> None:
216     """プランの月間上限を超えていればHTTP 429で拒否する。"""
217     plan = user["plan"]
218     monthly_limit = PLAN_MONTHLY_LIMITS.get(
219         plan,
220         PLAN_MONTHLY_LIMITS["free"],
221     )
222     monthly_count = get_monthly_usage_count(user["id"])
223
224     if monthly_count >= monthly_limit:
225         raise HTTPException(
226             status_code=status.HTTP_429_TOO_MANY_REQUESTS,
227             detail={
228                 "message": "Monthly compile limit exceeded",
229                 "plan": plan,
230                 "limit": monthly_limit,
231                 "used": monthly_count,
232             },
233         )
234
235
236 # -----
237 # P2コンパイルサーバーとの通信
238 # -----
239
240 def send_typeset_request(payload: dict) -> dict:
241     """内部トークン付きでP2へ要求し、JSON応答を返す。"""
242     try:
243         response = requests.post(
244             P2_TYPESET_URL,
245             headers={"Authorization": f"Bearer {P2_API_TOKEN}"},
246             json=payload,
247             timeout=P2_REQUEST_TIMEOUT_SECONDS,
248         )
249     except requests.RequestException as error:
250         raise HTTPException(
251             status_code=status.HTTP_502_BAD_GATEWAY,
252             detail=f"P2 connection failed: {error}",
253         ) from error
254
255     if response.status_code != status.HTTP_200_OK:
256         # P2の独自エラー形式 {"message": "..."} と FastAPI標準形式
257         # {"detail": "..."} の両方から本文だけを取り出す。
258         p2_detail = response.text
259         try:
260             p2_error = response.json()
261             if isinstance(p2_error, dict):
262                 detail = p2_error.get("message") or p2_error.get("detail")
263                 if isinstance(detail, str) and detail:
264                     p2_detail = detail
265         except ValueError:
266             pass
267
268         raise HTTPException(
269             status_code=status.HTTP_502_BAD_GATEWAY,
270             detail={
271                 "message": "P2 compile failed",
272                 "p2_status": response.status_code,
273                 "p2_detail": p2_detail,
274             },
275         )
276
277     try:
278         result = response.json()
279     except ValueError as error:
280         raise HTTPException(
281             status_code=status.HTTP_502_BAD_GATEWAY,
282             detail="P2 returned an invalid JSON response",
283         ) from error
284
285     if not isinstance(result, dict):

```

```

286         raise HTTPException(
287             status_code=status.HTTP_502_BAD_GATEWAY,
288             detail="P2 returned an unexpected response",
289         )
290
291     return result
292
293
294 def make_typeset_payload(request: CompileRequest) -> dict:
295     """P1の入力モデルをP2 /v1/typeset用のJSONへ変換する。"""
296     return {
297         "cardID": str(uuid.uuid4()),
298         "engine": request.engine,
299         "source": request.source,
300         "pictures": [item.model_dump() for item in request.pictures],
301         "files": [item.model_dump() for item in request.files],
302     }
303
304
305 # -----
306 # 公開API
307 # -----
308
309 @app.post("/login")
310 def login(request: LoginRequest):
311     """メールアドレスとパスワードを検証してJWTを返す。"""
312     user = authenticate_user(request.email, request.password)
313     if user is None:
314         raise unauthorized("Invalid email or password")
315
316     return {
317         "access_token": create_access_token(user["id"], user["email"]),
318         "token_type": "bearer",
319     }
320
321
322 @app.get("/me")
323 def me(user=Depends(get_current_user)):
324     """JWTに対応する現在のユーザー情報を返す。"""
325     plan = user["plan"]
326     return {
327         "service": SERVICE_NAME,
328         "id": user["id"],
329         "email": user["email"],
330         "plan": plan,
331         "status": user["status"],
332         "used": get_monthly_usage_count(user["id"]),
333         "limit": PLAN_MONTHLY_LIMITS.get(
334             plan,
335             PLAN_MONTHLY_LIMITS["free"],
336         ),
337     }
338
339
340 @app.post("/compile-test")
341 def compile_test(user=Depends(get_current_user)):
342     """固定の最小文書でP1からP2までの経路を確認する。"""
343     payload = {
344         "cardID": str(uuid.uuid4()),
345         "engine": "lualatex",
346         "source": r"""
347 \documentclass{article}
348 \begin{document}
349 Hello from P1.
350 \end{document}
351 """,
352         "pictures": [],
353         "files": [],
354     }
355     result = send_typeset_request(payload)
356
357     return {
358         "message": "compile succeeded",
359         "user_id": user["id"],
360         "pdf_received": bool(result.get("pdfBase64")),
361         "log": result.get("log", ""),
362     }
363
364
365 @app.post("/compile")
366 def compile_tex(
367     request: CompileRequest,
368     user=Depends(get_current_user),
369 ):
370     """利用上限を確認し、ユーザーのTeX文書をP2でコンパイルする。"""
371     ensure_monthly_limit(user)
372
373     start_time = time.monotonic()
374     input_bytes = len(request.source.encode("utf-8"))

```



```
375     success = False
376
377     try:
378         result = send_typeset_request(make_typeset_payload(request))
379         success = True
380         return {
381             "user_id": user["id"],
382             "pdfBase64": result.get("pdfBase64"),
383             "log": result.get("log", ""),
384         }
385     finally:
386         # P2接続失敗やコンパイル失敗も1回の要求として記録する。
387         record_usage(
388             user_id=user["id"],
389             compile_seconds=time.monotonic() - start_time,
390             input_bytes=input_bytes,
391             success=success,
392         )
```

主要な設定は次のとおりである。

設定	現行値・意味
JWT 方式	HS256
JWT 有効期限	30 分
JWT claim	sub, email, iat, exp
P2 URL	TEXNOTE_P2_TYPESET_URL。未設定時は http://192.168.3.4:8000/v1/typeset
P2 HTTP timeout	30 秒
HTTP リクエスト上限	32 MiB (33,554,432 bytes)
月間上限	free 50、standard 1000、pro 5000

P1 は Content-Length を確認し、32 MiB を超える要求を Pydantic による解析や P2 への転送より前に HTTP 413 で拒否する。不正な Content-Length は HTTP 400 とする。上限超過時に TeXNote へ返すメッセージは次のとおりである。

```
リクエストの上限 32 MiB を超えています。
```

5.3 起動

```
cd ~/tex-auth
source .auth-env/bin/activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

正常時は Uvicorn running on http://0.0.0.0:8000 と表示される。終了は Ctrl-C で行う。開発中は http://P1-IP:8000/docs の Swagger UI も利用できる。

6 ログインと保護 API の試験

6.1 POST /login

```
curl -sS -X POST http://127.0.0.1:8000/login \
-H 'Content-Type: application/json' \
-d '{
    "email": "test@example.com",
    "password": "<登録したパスワード>"
}'
```

成功時は次の形式で JWT が返る。

```
{"access_token":"eyJ...", "token_type":"bearer"}
```

実際の操作記録では成功応答と、誤ったパスワードに対する 401 Invalid email or password の両方を確認した。JWT 実値は文書へ保存しない。

6.2 GET /me

```
curl -sS http://127.0.0.1:8000/me \
-H 'Authorization: Bearer <USER_JWT>'
```

```
{"id":1, "email":"test@example.com", "plan":"free", "status":"active", "used":0, "limit":50}
```

偽 JWT、期限切れ JWT、Bearer ヘッダーなしも試し、拒否されることを確認する。get_current_user() は JWT を検証した後に users 表を再検索するため、ユーザーを inactive にすれば発行済み JWT の期限内でもアクセスを停止できる。

7 P2 との疎通と内部認証

7.1 ネットワークとヘルスチェック

P1 上で実行する。

```
curl -sS http://192.168.3.4:8000/health
```

```
{
  "status": "ok",
  "engines": [
    "lualatex", "xelatex", "pdflatex", "uplatex", "platex"
  ]
}
```

7.2 P1-P2 固定トークン

```
curl -sS http://192.168.3.4:8000/v1/auth-check \
-H "Authorization: Bearer $TEXNOTE_API_TOKEN"
```

"status":"ok"が返れば、LAN 経路と内部トークンが正しい。USER_JWT をここで使わず、逆に TEXNOTE_API_TOKEN をユーザーへ渡さない。

8 コンパイル中継の段階的試験

8.1 認証だけの compile-test

構築途中では、最初に次の最小実装で保護 API だけを確認した。

ソースコード 10 認証だけを確認する初期 compile-test

```
1 @app.post("/compile-test")
2 def compile_test(user=Depends(get_current_user)):
3     return {
4         "message": "compile allowed",
```

```

5     "user_id": user["id"]
6 }

```

JWT ありで成功し、Bearer なし・偽 JWT で拒否されることを確認してから P2 中継を追加した。

8.2 現行 compile-test

現行版は固定の最小 LuaLaTeX ソースを P2 へ送る。P1 の JWT 認証、P1-P2 内部認証、P2 版組を一度に確認できる。

```

curl -sS -X POST http://127.0.0.1:8000/compile-test \
-H 'Authorization: Bearer <USER_JWT>'

```

実際の試験では compile succeeded、pdf_received: true、P2 ログの Output written on main.pdf を確認した。

8.3 POST /compile

```

curl -sS -X POST http://127.0.0.1:8000/compile \
-H 'Authorization: Bearer <USER_JWT>' \
-H 'Content-Type: application/json' \
-d '{
    "engine": "lualatex",
    "source": "\\documentclass{article}\\begin{document}Hello\\end{document}",
    "pictures": [],
    "files": []
}'

```

添付要素の形式は次のとおりである。

```

{
  "relativePath": "Cards/<CARD-UUID>/files/python/tokens.py",
  "data": "<BASE64_DATA>"
}

```

P1 は relativePath を変更せず、サブフォルダ階層を維持したまま P2 へ中継する。また、cardID を UUID で生成し、P2 の /v1/typeset 用 JSON へ変換する。成功時は user_id、pdfBase64、log を返す。実際の試験では P2 の LuaLaTeX が 1 ページの PDF を生成し、Base64 が P1 経由で返るところまで確認済みである。

8.4 P2 エラー詳細の中継

P2 が非 200 応答を返した場合、P1 は P2 独自形式の message または FastAPI 標準形式の detail から本文だけを取り出す。P1 は HTTP 502 の detail へ message、p2_status、p2_detail を格納して TeXNote へ返す。TeXNote 側は p2_detail を優先表示するため、JSON の外枠は表示されず、文字列中の \n は実際の改行になる。これにより、P2 が返した TeX ログのファイル名、行番号、エラー内容を iPad 版・iPhone 版・macOS 版で確認できる。

9 利用履歴と月間回数制限

record_usage() は P2 要求の経過時間、TeX source の UTF-8 バイト数、成功・失敗を usage 表へ記録する。P2 への接続失敗と P2 の非 200 応答も失敗履歴として残す。現在の input_bytes には pictures/files のデータ量を

含まない。

現行の show_usage.py 全文を示す。

ソースコード 11 tools/show_usage.py：利用履歴確認

```
1  """コンパイル利用履歴と当月のユーザー別要求数を表示する。"""
2
3  from collections.abc import Iterable
4  import sqlite3
5
6  from database import get_connection
7
8
9  def fetch_usage_history() -> list[sqlite3.Row]:
10     """新しい順に全コンパイル履歴を取得する。"""
11     with get_connection() as connection:
12         return connection.execute(
13             """
14             SELECT
15                 id,
16                 user_id,
17                 created_at,
18                 compile_seconds,
19                 input_bytes,
20                 success
21             FROM usage
22             ORDER BY id DESC
23             """
24         ).fetchall()
25
26
27  def fetch_monthly_counts() -> list[sqlite3.Row]:
28     """当月に受け付けた要求数をユーザーごとに取得する。"""
29     with get_connection() as connection:
30         return connection.execute(
31             """
32             SELECT user_id, COUNT(*) AS count
33             FROM usage
34             WHERE strftime('%Y-%m', created_at) =
35                   strftime('%Y-%m', 'now')
36             GROUP BY user_id
37             ORDER BY user_id
38             """
39         ).fetchall()
40
41
42  def print_rows(title: str, rows: Iterable[sqlite3.Row]) -> None:
43     """見出しと SQLite 行の一覧を辞書形式で表示する。"""
44     print(f"{title}:")
45     for row in rows:
46         print(dict(row))
47
48
49  def main() -> None:
50     """利用履歴と当月集計を続けて表示する。"""
51     print_rows("Usage history", fetch_usage_history())
52     print_rows("Monthly counts", fetch_monthly_counts())
53
54
55  if __name__ == "__main__":
56     main()
```

```
cd ~/tex-auth
source .auth-env/bin/activate
python -m tools.show_usage
```

plan	月間要求回数
free	50
standard	1000
pro	5000

新規登録と管理コマンドは未定義 plan を拒否する。既存 DB に未定義 plan が残っている場合だけ安全側として free の上限を適用する。成功・失敗を問わず usage の 1 行を 1 要求として数え、上限以上なら P2 へ送る前に HTTP 429 を返す。

10 現行ソースを配置する方法

リポジトリから P1 へ転送する例を示す。秘密値と DB はこの操作に含めない。

```
# Mac の TeXNote リポジトリ直下で実行
scp Server/P1/progs/main.py \
    Server/P1/progs/auth.py \
    Server/P1/progs/database.py \
    Server/P1/progs/plans.py \
    mat@192.168.3.21:/home/mat/tex-auth/

scp -r Server/P1/progs/tools \
    mat@192.168.3.21:/home/mat/tex-auth/
```

P1 上で構文を確認する。main.py の import には両環境変数が必要である。

```
cd ~/tex-auth
source .auth-env/bin/activate
python -m py_compile *.py tools/*.py
```

11 ソース整理で改善した点と残る課題

11.1 main.py の整理と修正

現行 main.py は、設定、入出力モデル、JWT、利用制限、P2 通信、公開 API の順に構成を整理した。P2 通信の重複を `send_typeset_request()` へ集約し、各関数へ型注釈、docstring、日本語コメントを追加した。また、次の点を修正している。

- SQLite で無効だった `# AND success = 1` を月間集計 SQL から削除した。
- `pictures/files` の既定値を `Field(default_factory=list)` へ変更した。
- `engine` を P2 が対応する 5 種類の `Literal` へ制限した。
- JWT の `sub` が欠落または整数変換不能の場合も 401 として処理する。
- P2 の不正 JSON と想定外の JSON 型を 502 として処理する。
- P1 入口の HTTP リクエストを 32 MiB へ制限した。
- P2 の `message/detail` を抽出し、`p2_detail` として中継する。
- P2 URL を環境変数で上書き可能にした。
- 成功・失敗の `usage` 記録を `finally` で一箇所に集約した。

11.2 その他の Python ソースの整理

`database.py`、`auth.py`、`plans.py`、`tools` 配下も main.py と同じ書式へ統一した。各ファイルにモジュール説明、各関数に docstring と型注釈、判断理由を示す日本語コメントを追加し、変数名を `con` や `row` から役割が分かる名前へ変更した。主な改善点は次のとおりである。

- DB 接続をコンテキストマネージャー化し、処理後に必ず閉じる。
- `useradd` の入力処理と登録処理を関数へ分離する。
- `plan` 名と月間上限を `plans.py` へ集約し、未定義 `plan` の登録を拒否する。
- `change_plan` で既存ユーザーの `plan` を検証して変更する。
- `check01/check02` の DB 検索を再利用可能な関数へ分離する。

- show_usage の履歴取得、月間集計、表示処理を分離する。
- init_db の各テーブルについて用途をコメントで説明する。

11.3 運転用と保守用の分離

サーバー運転に必要な main.py、auth.py、database.py、plans.py はルートへ、DB 初期化、ユーザー追加、プラン変更、DB 確認、利用履歴表示は tools へ分離した。保守ツールは `python -m tools. コマンド名` でのみ明示的に実行し、Uvicorn の運転には読み込まれない。DB 本体は引き続き Git へ入れない。

11.4 公開前に追加する制限

- source 文字数、添付数、Base64 サイズ、相対パスを検証する。
- P1 の同時実行数とログイン試行回数を制限する。
- P2 の詳細エラーと TeX ログを外部へ返す範囲を見直す。
- usage の DB 書き込み失敗を API 本体のエラーと分けて監視する。

12 systemd による常時起動

P1 は、Raspberry Pi の起動時に自動的に立ち上がるよう systemd へ登録する。現行構成では、プログラムを `/home/mat/tex-auth`、Python 仮想環境を `/home/mat/tex-auth/.auth-env` へ配置し、mat ユーザーで Uvicorn を実行する。

12.1 Python 環境の確認

systemd へ登録する前に仮想環境を有効化し、P1 が必要とするパッケージを確認する。

```
cd /home/mat/tex-auth
source .auth-env/bin/activate
python -m pip show fastapi uvicorn pyjwt argon2-cffi requests
```

実際の確認時には、FastAPI 0.141.1、Uvicorn 0.52.3、PyJWT 2.13.0、argon2-cffi 25.1.0、requests 2.34.2 が仮想環境内に導入されていた。各パッケージの Location が `/home/mat/tex-auth/.auth-env/lib/python3.11/site-packages` 以下を指すことも確認する。

次に、systemd が使用する Uvicorn の実体を確認する。

```
ls -l /home/mat/tex-auth/.auth-env/bin/uvicorn
```

12.2 手動起動の確認

systemd の問題とアプリケーション自身の問題を区別できるよう、最初に同じ仮想環境から手動起動を確認する。秘密値は実際の値へ置き換える。

```
cd /home/mat/tex-auth
source .auth-env/bin/activate
export TEX_AUTH_SECRET_KEY='<JWT 署名鍵>'
export TEXNOTE_API_TOKEN='<P2 と共通の内部トークン>'
export TEXNOTE_P2_TYPESET_URL='http://192.168.3.4:8000/v1/typeset'
uvicorn main:app --host 0.0.0.0 --port 8000
```

Application startup complete. および Uvicorn running on http://0.0.0.0:8000 が表示されたら、別の端末から応答を確認する。

```
curl -I http://127.0.0.1:8000/docs
```

確認後は Ctrl+C で手動起動した Uvicorn を停止する。

12.3 配置と権限の確認

P1 は mat ユーザーで動かすため、プログラムと SQLite DB を同ユーザーが扱えるようにする。

```
sudo chown -R mat:mat /home/mat/tex-auth
chmod 700 /home/mat/tex-auth
chmod 600 /home/mat/tex-auth/tex_service.db
```

12.4 秘密値ファイルの作成

秘密値は Python ソースや systemd サービスファイルへ直接書かず、root だけが読み書きできる環境ファイルへ保存する。

```
sudo install -o root -g root -m 600 /dev/null /etc/tex-auth.env
sudoedit /etc/tex-auth.env
```

内容は次のとおりである。TEXNOTE_API_TOKEN には P2 と同じ内部トークンを設定し、TEX_AUTH_SECRET_KEY とは必ず別の値にする。

```
TEX_AUTH_SECRET_KEY=<JWT 署名鍵>
TEXNOTE_API_TOKEN=<P2 内部トークン>
TEXNOTE_P2_TYPESET_URL=http://192.168.3.4:8000/v1/typeset
```

JWT 署名鍵を新しく生成する場合は次のコマンドを使用できる。

```
python3 -c 'import secrets; print(secrets.token_urlsafe(64))'
```

環境ファイルが root:root、モード 600 であることを確認する。

```
sudo ls -l /etc/tex-auth.env
```

12.5 サービスファイルの作成

/etc/systemd/system/tex-auth.service を作成する。

```
sudoedit /etc/systemd/system/tex-auth.service
```

内容は次のとおりである。ExecStart はサービスファイル内では 1 行で記述する。

```
[Unit]
Description=TeXNote P1 Authentication API
Wants=network-online.target
After=network-online.target
```

```
[Service]
Type=simple
User=mat
Group=mat
WorkingDirectory=/home/mat/tex-auth
EnvironmentFile=/etc/tex-auth.env
ExecStart=/home/mat/tex-auth/.auth-env/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

12.6 設定検査と自動起動の有効化

サービスファイルの構文を検査する。問題がなければ通常は何も表示されない。

```
sudo systemd-analyze verify /etc/systemd/system/tex-auth.service
```

systemd へ設定を読み込ませ、直ちに起動すると同時に OS 起動時の自動起動を有効にする。

```
sudo systemctl daemon-reload
sudo systemctl enable --now tex-auth.service
systemctl status tex-auth.service --no-pager -l
journalctl -u tex-auth.service -n 100 --no-pager
```

実際の確認では、Loaded 欄が enabled、Active 欄が active (running) となり、Uvicorn の Application startup complete. が記録された。

12.7 API と再起動後の確認

P1 自身から API ドキュメントの応答を確認する。

```
curl -I http://127.0.0.1:8000/docs
```

最後に Raspberry Pi を再起動し、systemd によって P1 が自動起動することを確認する。

```
sudo reboot
```

再接続後に次を実行する。

```
systemctl is-enabled tex-auth.service
systemctl is-active tex-auth.service
systemctl status tex-auth.service --no-pager -l
```

先頭 2 コマンドがそれぞれ enabled、active を返せば、自動起動は正常である。

12.8 運用時の操作

ソースを更新した場合はサービスを再起動し、直後のログを確認する。


```
sudo systemctl restart tex-auth.service
journalctl -u tex-auth.service -n 100 --no-pager
```

サービスの起動、停止、再起動には次を使用する。

```
sudo systemctl start tex-auth.service
sudo systemctl stop tex-auth.service
sudo systemctl restart tex-auth.service
```

環境ファイルまたはサービスファイルを変更した場合は、設定を再読み込みしてから再起動する。

```
sudo systemctl daemon-reload
sudo systemctl restart tex-auth.service
```

`main.py` は起動時に `TEX_AUTH_SECRET_KEY` と `TEXNOTE_API_TOKEN` を必須取得する。設定漏れや記述誤りがある場合はサービスが起動しないため、`journalctl` で原因を確認する。

13 Docker 版への移行

ここまでは、P1 をホスト上の Python 仮想環境から systemd で起動する構成を説明した。本節では、同じ P1 を Docker Compose で運転する手順を示す。Docker 版の検査が完了するまで systemd 版のプログラム、仮想環境、サービスファイル、DB は削除しない。

13.1 移行方針

Docker 版は次の構成とする。

- arm64 対応の `python:3.11-slim-bookworm` を使用する。
- 運転用の `main.py`、`auth.py`、`database.py`、`plans.py` をイメージへ入れる。
- 保守用の `tools` は DB 初期化やユーザー管理のためイメージへ入れるが、常時実行しない。
- JWT 署名鍵と P2 内部トークンはイメージへ入れず、`p1.env` から起動時に渡す。
- SQLite DB はホストの `data` ディレクトリへ保存し、コンテナを作り直しても残す。
- コンテナ内では `root` ではなく UID 10001 の専用ユーザー `texauth` で実行する。
- イメージ本体を読み取り専用にし、DB 用/`data` と一時領域/`tmp` だけを書き込み可能にする。
- systemd 版と Docker 版は同じ TCP 8000 を同時に使用しない。

13.2 SQLite 保存先を環境変数化する

現行の `database.py` は DB をソースと同じディレクトリへ固定している。このままイメージを読み取り専用にすると、SQLite の DB 本体や `journal` を書き込めない。そこで `TEX_AUTH_DB_PATH` が設定されている場合だけ、その場所を使用するようにする。

ソースコード 12 Docker の永続領域へ対応する `database.py` の DB パス定義

```
1 import os
2 from pathlib import Path
3
4 DEFAULT_DB_PATH = Path(__file__).resolve().parent / "tex_service.db"
5 DB_PATH = Path(os.getenv("TEX_AUTH_DB_PATH", DEFAULT_DB_PATH))
```

既存の `from pathlib import Path` は残し、`import os` を追加する。既存の `DB_PATH` 定義は上記 2 行へ置き換え、同じ役割の定義を重複して残さない。systemd 版では環境変数を設定しないため、従来どおり `/home/mat/tex-auth/tex_service.db` を使用する。

13.3 Docker Engine と Compose の確認

P2 ですでに Docker を使用している場合と同様、P1 でも Docker Engine と Compose plugin を使う。未導入の場合は本書の P2 Docker 版と同じ公式 Debian 用 APT リポジトリから導入する。

```
sudo systemctl is-enabled docker
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
```

本手順では Docker へ実質的な `root` 権限があることを明示するため、すべて `sudo docker` で実行する。

13.4 Docker 用ディレクトリの準備

systemd 版と混同しないよう、P1 上へ別ディレクトリを作成する。

```
mkdir -p /home/mat/tex-auth-docker/data
chmod 700 /home/mat/tex-auth-docker
```

Mac から現行ソースを転送する。

```
scp -r /Users/mat/Documents/Git/TeXNote/Server/P1/progs/* \
  mat@192.168.3.21:/home/mat/tex-auth-docker/
```

__pycache__や既存 DB はイメージへ入れない。現行 DB は停止時に整合した状態でコピーするため、この時点ではまだコピーしない。

13.5 requirements.txt の作成

/home/mat/tex-auth-docker/requirements.txt を次の内容で作成する。実際に仮想環境で確認した直接依存パッケージの版を固定する。

```
fastapi==0.141.1
uvicorn[standard]==0.52.3
PyJWT==2.13.0
argon2-cffi==25.1.0
requests==2.34.2
```

13.6 Dockerfile の作成

/home/mat/tex-auth-docker/Dockerfile を次の内容で作成する。

```
FROM python:3.11-slim-bookworm

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

WORKDIR /app

COPY requirements.txt ./
RUN python -m pip install --no-cache-dir -r requirements.txt

COPY main.py auth.py database.py plans.py ./
COPY tools ./tools

RUN useradd --system --uid 10001 --create-home texauth \
    && mkdir -p /data \
    && chown texauth:texauth /data

USER texauth

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD python -c "import urllib.request;
    ↪ urllib.request.urlopen('http://127.0.0.1:8000/openapi.json', timeout=4)" || exit 1
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

13.7 秘密情報をビルド対象から除外する

/home/mat/tex-auth-docker/.dockerignore を作成する。

```
.git
.DS_Store
.auth-env
__pycache__
*.pyc
*.db
*.db-journal
*.db-shm
*.db-wal
p1.env
data
docs
out
```

これにより、JWT 署名鍵、P2 内部トークン、ユーザー DB、利用履歴が build context やイメージ layer へ入ることを防ぐ。

13.8 Docker 用環境ファイルの作成

systemd 版の秘密値を画面へ表示せず、Docker 用環境ファイルへ root 権限でコピーする。

```
sudo install -o root -g root -m 600 \
  /etc/tex-auth.env \
  /home/mat/tex-auth-docker/p1.env
sudoedit /home/mat/tex-auth-docker/p1.env
```

次の 4 項目を設定する。最初の 2 項目は systemd 版と同じ値を使用する。

```
TEX_AUTH_SECRET_KEY=<systemd 版と同じ JWT 署名鍵>
TEXNOTE_API_TOKEN=<P2 と共通の内部トークン>
TEXNOTE_P2_TYPESET_URL=http://192.168.3.4:8000/v1/typeset
TEX_AUTH_DB_PATH=/data/tex_service.db
```

13.9 Compose 設定の作成

/home/mat/tex-auth-docker/compose.yaml を作成する。P1 の LAN 側 IP が 192.168.3.21 でない場合は、ports を実機に合わせる。

```
services:
  tex-auth:
    build:
      context: .
      dockerfile: Dockerfile
    image: tex-auth:bookworm
    restart: unless-stopped
```

```

env_file:
  - ./p1.env
ports:
  - "192.168.3.21:8000:8000"
volumes:
  - ./data:/data
read_only: true
tmpfs:
  - /tmp:rw,noexec,nosuid,size=64m,mode=1777
security_opt:
  - no-new-privileges:true
cap_drop:
  - ALL
pids_limit: 128

```

設定を検査する。引数なしの `docker compose config` は秘密値を展開表示する可能性があるため使用しない。

```

cd /home/mat/tex-auth-docker
sudo docker compose config --quiet

```

13.10 イメージのビルドと静的検査

ビルドは `systemd` 版を動かしたまま実行できる。

```

cd /home/mat/tex-auth-docker
df -h /
sudo docker compose build
sudo docker image ls tex-auth:bookworm

```

まだポートを公開せず、イメージ内の Python、ファイル、実行ユーザーを確認する。

```

sudo docker compose run --rm --no-deps tex-auth \
  sh -lc 'python --version; id; ls -l /app'

```

13.11 systemd 版 DB の移行

SQLite DB をコピーする間に書き込みが発生しないよう、`systemd` 版を停止する。この操作以降、検査が終わるまで P1 API は一時停止する。

```

sudo systemctl stop tex-auth.service
systemctl is-active tex-auth.service

```

DB 本体を Docker の永続領域へコピーし、コンテナ内ユーザー UID 10001 が読み書きできるようにする。

```

sudo install -o 10001 -g 10001 -m 600 \
  /home/mat/tex-auth/tex_service.db \
  /home/mat/tex-auth-docker/data/tex_service.db
sudo chown 10001:10001 /home/mat/tex-auth-docker/data
sudo chmod 700 /home/mat/tex-auth-docker/data
sudo ls -ln /home/mat/tex-auth-docker/data

```

コンテナから DB を開けることと、3 表が存在することを確認する。

```
cd /home/mat/tex-auth-docker
sudo docker compose run --rm --no-deps tex-auth \
  python -m tools.check01
```

13.12 systemd 版から Docker 版への切替

systemd 版の自動起動を無効にする。サービスファイルと従来の DB は削除しない。

```
sudo systemctl disable tex-auth.service
systemctl is-enabled tex-auth.service
```

Docker 版をバックグラウンドで起動する。

```
cd /home/mat/tex-auth-docker
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs --tail=100 tex-auth
```

コンテナ状態を確認する。

```
sudo docker compose ps
sudo docker inspect \
  --format '{{.State.Status}} {{if .State.Health}}{{.State.Health.Status}}{{end}}' \
  tex-auth-docker-tex-auth-1
curl --fail-with-body http://127.0.0.1:8000/openapi.json >/dev/null
```

Compose のディレクトリ名を変えた場合はコンテナ名も変わる。正確な名前は `sudo docker compose ps` で確認する。

13.13 認証と P2 中継の確認

登録済みユーザーでログインし、返された JWT で /me を確認する。

```
curl -sS -X POST http://127.0.0.1:8000/login \
  -H 'Content-Type: application/json' \
  -d '{"email": "<登録済みメール>", "password": "<パスワード>"}'
```

JWT を端末変数へ設定して確認する。

```
USER_JWT='<login で返された access_token>'
curl --fail-with-body http://127.0.0.1:8000/me \
  -H "Authorization: Bearer $USER_JWT"
curl --fail-with-body -X POST http://127.0.0.1:8000/compile-test \
  -H "Authorization: Bearer $USER_JWT"
unset USER_JWT
```

/compile-test が成功すれば、Docker 版 P1 から P2 への通信、P2 内部トークン、P2 での TeX コンパイルまで確認できる。続いて macOS 版、iPad 版、iPhone 版から実際にログインし、任意の Card を版組する。

13.14 再起動後の自動復旧試験

Docker サービスと Compose の restart: unless-stopped による自動復旧を確認する。

```
sudo reboot
```

再接続後に次を実行する。

```
sudo systemctl is-active docker
cd /home/mat/tex-auth-docker
sudo docker compose ps
curl --fail-with-body http://127.0.0.1:8000/openapi.json >/dev/null
```

13.15 更新と日常運用

Python ソースまたは requirements.txt を更新した場合は、ファイルを転送してイメージを再ビルドする。単なる restart ではイメージ内のソースは更新されない。

```
cd /home/mat/tex-auth-docker
sudo docker compose config --quiet
sudo docker compose up -d --build
sudo docker compose ps
sudo docker compose logs --tail=100 tex-auth
```

日常の状態確認とログ確認には次を使用する。

```
cd /home/mat/tex-auth-docker
sudo docker compose ps
sudo docker compose logs --tail=100 tex-auth
sudo docker compose logs -f tex-auth
sudo docker stats --no-stream
```

ログ追跡は Ctrl+C で終了でき、コンテナは停止しない。明示的な停止と再開は次のとおり。

```
sudo docker compose stop
sudo docker compose start
```

13.16 DB のバックアップ

整合したバックアップを得るため、SQLite の backup API をコンテナ内から実行する。

```
cd /home/mat/tex-auth-docker
sudo docker compose exec tex-auth python -c \
    "import sqlite3; src=sqlite3.connect('/data/tex_service.db');
    ↪ dst=sqlite3.connect('/data/tex_service.backup.db'); src.backup(dst); dst.close();
    ↪ src.close()"
sudo cp -p data/tex_service.backup.db \
    data/tex_service.backup-$(date +%Y%m%d-%H%M%S).db
```

バックアップには認証情報と利用履歴が含まれるため、権限 600 を維持し、Git へ登録しない。

13.17 問題発生時に systemd 版へ戻す

Docker 版で問題が発生した場合は、Docker 版を停止してから systemd 版へ戻す。両方を同時に起動すると TCP 8000 が競合する。

Docker 版で更新された DB を systemd 版へ戻す場合は、双方を停止した状態で退避とコピーを行う。

```
cd /home/mat/tex-auth-docker
sudo docker compose down
sudo cp -p /home/mat/tex-auth/tex_service.db \
    /home/mat/tex-auth/tex_service.db.before-docker-restore
sudo install -o mat -g mat -m 600 \
    data/tex_service.db \
    /home/mat/tex-auth/tex_service.db
sudo systemctl enable --now tex-auth.service
systemctl is-enabled tex-auth.service
systemctl is-active tex-auth.service
curl --fail-with-body http://127.0.0.1:8000/openapi.json >/dev/null
```

13.18 Docker 版完了チェックリスト

- ☐ Docker Engine と Compose plugin が動作している。
- ☐ database.py が TEX_AUTH_DB_PATH へ対応した。
- ☐ p1.env、SQLite DB、バックアップがイメージへ入っていない。
- ☐ コンテナが root ではなく UID 10001 で動作している。
- ☐ systemd 版を停止してから Docker 版を起動した。
- ☐ /login と /me が成功した。
- ☐ /compile-test で P2 中継と PDF 生成を確認した。
- ☐ macOS 版、iPad 版、iPhone 版からログインと版組を確認した。
- ☐ Raspberry Pi 再起動後にコンテナが自動復旧した。
- ☐ Docker 版 DB のバックアップと systemd 版への復旧手順を確認した。

14 セキュリティと公開前チェック

1. P2 を一般公開せず、P2 のファイアウォールは P1 からの接続だけを許可する。
2. P1 で HTTPS を終端し、パスワードと JWT を平文 HTTP で送らない。
3. 2 種類の秘密値を分離し、定期更新と安全な再起動方法を準備する。
4. DB とバックアップの権限を制限し、暗号化・復旧試験を行う。
5. P2 で shell-escape を禁止し、TeX を低権限・隔離環境で動かす。
6. login、JWT 期限切れ、inactive、月間上限、P2 停止、usage 記録を自動テストする。

15 再構築時の確認順序

1. P1 の OS、IP、時刻、Python 仮想環境を確認する。
2. SQLite の 3 表を作り、ユーザーを登録して Argon2 認証を試す。
3. 2 つの環境変数を設定する。
4. P1 から P2 の health と auth-check を確認する。
5. P1 を起動し、login、me を確認する。

6. compile-test、次に任意ソースの compile を確認する。
7. show_usage.py で成功・失敗履歴を確認する。
8. 月間上限、inactive、不正 JWT、P2 停止時の応答を確認する。
9. systemd 化し、再起動後も自動起動することを確認する。
10. HTTPS と P1/P2 のネットワーク制限を適用してから公開する。

16 まとめ

P1 は、Argon2 ユーザー認証、30 分 JWT、保護 API、P2 への固定トークン付き中継、PDF Base64 応答、利用履歴、月間回数制限を担当する。実際の操作ではユーザー認証、login、me、compile-test、実 compile、PDF 生成まで段階的に確認された。本書の順序で小さく確認し、各段階が成功してから次へ進むことで、認証、DB、LAN、P2 版組のどこに問題があるかを切り分けられる。